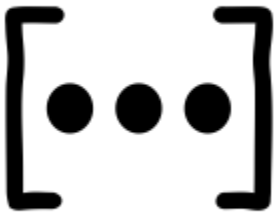
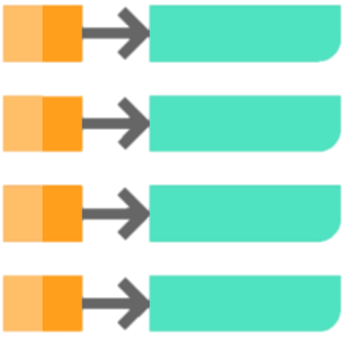
Longest Subarray With Equal 0s and 1s

Name	ID
AbdAllah Mahmoud AbdAllah	20240584
Omar Waleed Mohamed	20240661
Mohamed Adel Mohamed	20240848
Malak Mohamed Hussien	20240997
Menna Khaled Ismail	20241018
Heba Farag AbdElHamed	20241096





1- Understanding the Task:

contiguous subarray that contains an equal number of 0s and 1s. A contiguous subarray refers to a sequence of elements that appear consecutively in the original array without skipping any elements.

For example, in the array `[0, 0, 1, 0, 1, 1, 0]`, one of the longest balanced contiguous subarrays has a length of 6.

The key insight is that by transforming every 0 into -1 and every 1 into +1, the problem can be reduced to finding the longest subarray whose sum equals zero. This is because a balanced subarray will have its transformed values cancel each other out, resulting in a total sum of 0. Therefore, the original problem can be efficiently solved by searching for the longest zero-sum subarray after this transformation.

Main idea:

When the same prefix sum appears at two different positions, the elements between them must sum to zero, meaning there are equal numbers of 0s and 1s in that subarray.

Example:

Consider the following input array: nums = `[0, 0, 1, 0, 1, 1, 0]`
After conversion:

```
0 -> -1

1 -> +1
```

Converted array = `[-1, -1, +1, -1, +1, +1, -1]`

So, by using prefix sum, whenever a prefix sum value appears again, the subarray between the first occurrence and the current index is balanced. For this example, the longest balanced subarray length is 6.

2- First Algorithm: Non-Recursive:

The first algorithm follows an iterative approach. It traverses the array from left to right while maintaining a running prefix sum. the Java implementation uses two arrays: `firstIndex[]` and `visited[]`.

The `firstIndex[]` array stores the first index at which each prefix sum appears, while the `visited[]` array checks whether a prefix sum has already been encountered. Since prefix sums can have negative values, an offset equal to the array length is added to transform each prefix sum into a valid array index.

During traversal, if the current element is `0`, the prefix sum decreases by 1; otherwise, it increases by 1. When the same prefix sum appears again, it means the subarray between the two indices has a sum of zero, which indicates an equal number of 0s and 1s. The algorithm then calculates the subarray length by subtracting the first stored index from the current index.

2.1-Pseudo-code:

```
Input: Binary array nums

Output: Length of the longest contiguous subarray with equal 0s and 1s

Set n = nums.length

Set size = (2 * n) + 1

Create integer array firstIndex[size]

Create boolean array visited[size]

Set offset = n

Set firstIndex[offset] = -1

Set visited[offset] = true

Set prefixSum = 0

Set maxLength = 0

For i ← 0 to n - 1 do

{

    If nums[i] == 0 then

        prefixSum = prefixSum – 1
```

```
    Else
        prefixSum = prefixSum + 1
    Set position = prefixSum + offset
    If visited[position] == true then

        currentLength = i - firstIndex[position]
        maxLength = max(maxLength, currentLength)
    Else
        visited[position] = true
        firstIndex[position] = i
}
Return maxLength
```

2.2- Implementation:

```java
public class LongestBalancedSubarray {  2 usages
    public static int longestEqualZerosOnesNonRecursive(int[] nums) {  1 usage
        int n = nums.length; // O(1)
        int size = (2 * n) + 1; // O(1)
        int[] firstIndex = new int[size]; // O(n)
        boolean[] visited = new boolean[size]; // O(n)
        int offset = n; // O(1)
        firstIndex[offset] = -1; // O(1)
        visited[offset] = true; // O(1)
        int prefixSum = 0; // O(1)
        int maxLength = 0; // O(1)
        for (int i = 0; i < n; i++) { // O(n)

            if (nums[i] == 0) {
                prefixSum = prefixSum - 1;
            } else {
                prefixSum = prefixSum + 1;
            }
            int position = prefixSum + offset;
            if (visited[position]) {

                int currentLength =
                        i - firstIndex[position];

                if (currentLength > maxLength) {
                    maxLength = currentLength;
                }

            } else {

                visited[position] = true;
                firstIndex[position] = i;
            }
        }
        return maxLength; // O(1)
    }
```

// O(1)

2.3- Analysis and Complexity:

The algorithm processes an array of size `n` using a single loop that runs from i = 0 to n - 1. Therefore, the number of iterations is `n`.

Inside each iteration, a constant number of operations are performed: updating the prefix sum, calculating the adjusted array position using the offset, checking whether the position has been visited before, retrieving the first stored index if it exists, storing a new index if it does not exist, and updating the maximum length using a comparison operation.

All of these operations are array accesses, Boolean checks, arithmetic operations, or comparisons, which each take constant time:

$$O(1)$$

Therefore, the total cost of a single iteration is:

$$O(1)$$

Since the loop runs `n` times, the total running time is:

$$T(n) = n \cdot O(1)$$

Which simplifies to:

$$T(n) = O(n)$$

Final result:

$$O(n)$$

3- Second Algorithm: Recursive:

The second algorithm solves the same problem using a recursive approach instead of iteration. The function processes one element of the array at each recursive call. In each step, it updates the prefix sum, maps the prefix sum to a valid index using an offset, checks whether this prefix sum has been encountered before using auxiliary arrays (`firstIndex` and `visited`), and updates the maximum length of a balanced subarray if necessary. After completing these operations, the function recursively calls itself for the next index.

The recursion continues until the index reaches the length of the array, which acts as the base case and stops the recursive process.

3.1- Pseudo-code:

```
Input: Binary array nums

Output: Length of the longest contiguous subarray with equal 0s and 1s

Set n = length(nums)

Set size = (2 × n) + 1

Create integer array firstIndex[size]

Create boolean array visited[size]

Set offset = n

Set firstIndex[offset] = -1

Set visited[offset] = true

Function Solve(
    index,
    prefix_sum,
    max_length
)

{
    If index >= length(nums) then
        return max_length

    If nums[index] = 0 then
        prefix_sum = prefix_sum - 1
    Else
        prefix_sum = prefix_sum + 1
```

```
    position = prefix_sum + offset

    If visited[position] = true then
    {
        current_length =
            index - firstIndex[position]

        If current_length > max_length then
            max_length = current_length
    }

    Else
    {
        visited[position] = true

        firstIndex[position] = index
    }

    return Solve(
        index + 1,
        prefix_sum,
        max_length
    )
}

Return Solve(0, 0, 0)
```

3.2- Implementation:

```java
public static int longestEqualZerosOnesRecursive(int[] nums) {  1 usage
    int n = nums.length; // O(1)
    int size = (2 * n) + 1; // O(1)
    int[] firstIndex = new int[size]; // O(n)
    boolean[] visited = new boolean[size]; // O(n)
    int offset = n; // O(1)
    firstIndex[offset] = -1; // O(1)
    visited[offset] = true; // O(1)
    return solveRecursive(
            nums,
            index: 0,
            prefixSum: 0,
            maxLength: 0,
            firstIndex,
            visited,
            offset
    );
}
```

// O(1)

```java
private static int solveRecursive(  2 usages
        int[] nums,
        int index,
        int prefixSum,
        int maxLength,
        int[] firstIndex,
        boolean[] visited,
        int offset
) {
    if (index >= nums.length) {
        return maxLength;
    }
    if (nums[index] == 0) {
        prefixSum = prefixSum - 1;
    } else {
        prefixSum = prefixSum + 1;
    }
    int position = prefixSum + offset;
    if (visited[position]) {
        int currentLength =
                index - firstIndex[position];
        if (currentLength > maxLength) {
            maxLength = currentLength;
        }
    } else {

        visited[position] = true;
        firstIndex[position] = index;
    }
    return solveRecursive(
            nums,
            index: index + 1,
            prefixSum,
            maxLength,
            firstIndex,
            visited,
            offset
    );
```

// O(1)

3.3- Analysis and Time Complexity:

Let `n` be the length of the input array `nums`. The recursive algorithm processes the array starting from index `0` up to index `n - 1`. In each recursive call, the function processes exactly one element and then makes a single recursive call for the next index, reducing the remaining problem size by 1 each time.

This leads to the recurrence relation:

$$T(n) = T(n-1) + O(1)$$

At each recursive call, the algorithm performs a constant number of operations, including updating the prefix sum, mapping it to a valid index using the offset technique, checking whether the position has been visited using an array lookup, retrieving or inserting the first occurrence index, and updating the maximum length. Each of these operations takes constant time:

$$O(1)$$

Expanding the recurrence step by step:

$$T(n) = T(n-1) + 1$$
$$T(n) = T(n-2) + 2$$
$$T(n) = T(n-3) + 3$$

Continuing this expansion until reaching the base case:

$$T(n) = T(0) + n$$

Since `T(0)` is a constant, the final expression becomes:

$$T(n) = O(n)$$

Therefore, the recursive algorithm has linear time complexity because it processes each element exactly once and performs only constant-time operations at each step.

Final result:

$$O(n)$$

4- Comparison Between the Two Algorithms:

Aspect	Non-Recursive	Recursive
Main Concept	Scan once and track sums	Process recursively and track sums
Time Complexity	O(n)	O(n)
Reason Of Time Complexity	Single loop processes each element once with O(1) operations per loop.	One recursive call per element, each with O(1).
Memory Usage	Lower (no call stack).	Higher (recursive call stack is used)
Performance	Faster in Bigger Arrays (no function overhead).	Slightly slower due to recursive calls (may lead to function overhead).

5- Final Conclusion:

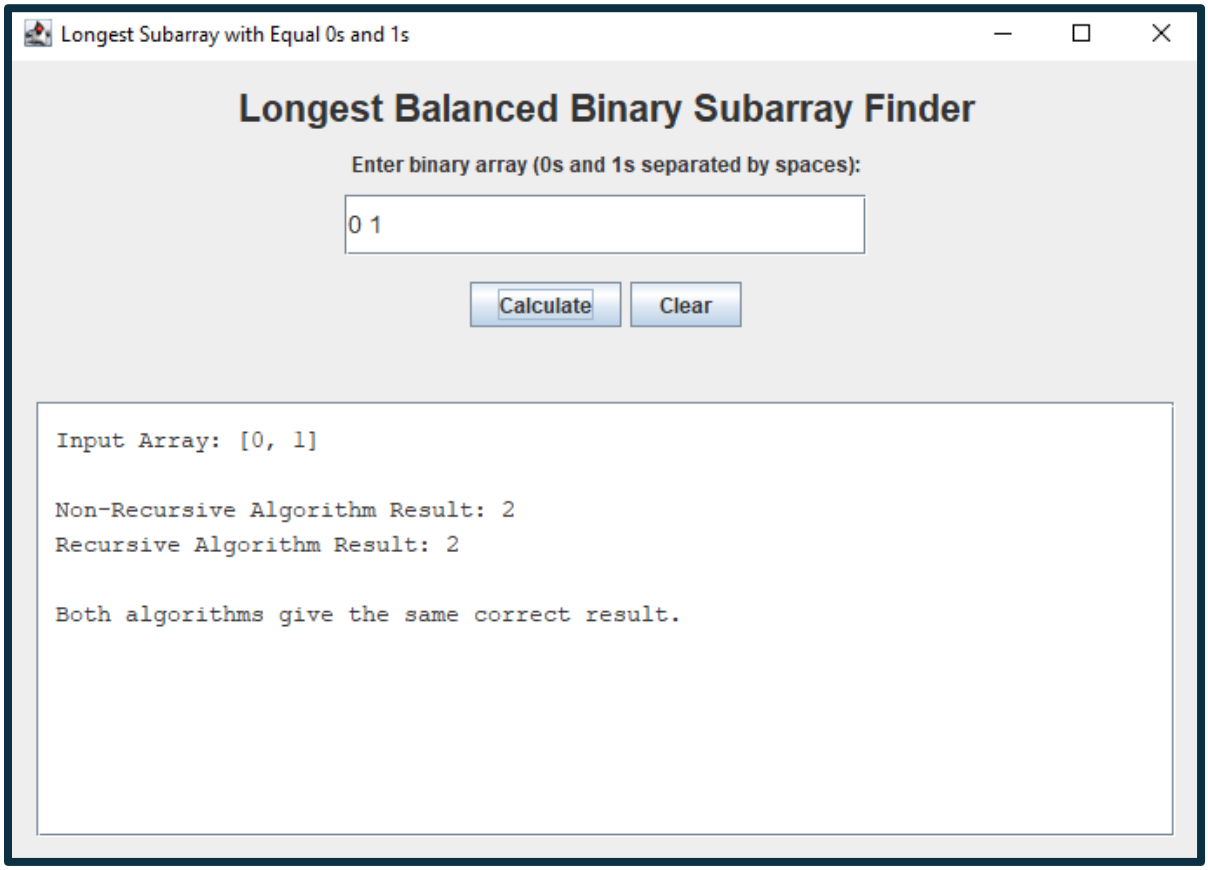
Both the non-recursive and recursive approaches solve the same problem using the same core idea: transforming the array values and maintaining a prefix sum to find the longest subarray with equal numbers of 0s and 1s. Instead of using dynamic key-value storage, both implementations use fixed-size auxiliary arrays (firstIndex[] and visited[]) with an offset technique to map prefix sum values into valid indices. As a result, both approaches share the same theoretical time and space complexities, both being O(n).

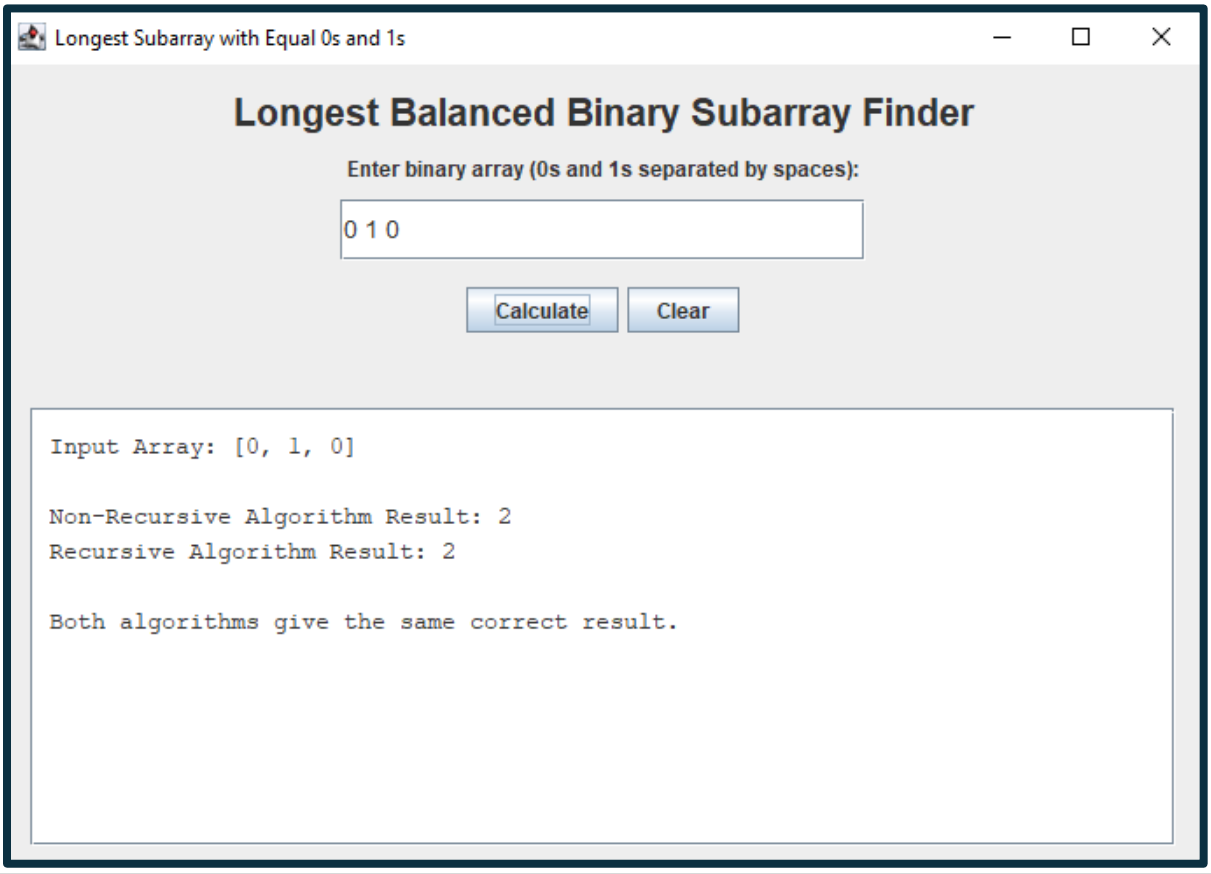
However, the main difference lies in implementation style and runtime behavior. The non-recursive approach uses a simple loop, making it more efficient in practice because it avoids function call overhead and does not use recursion stack memory.

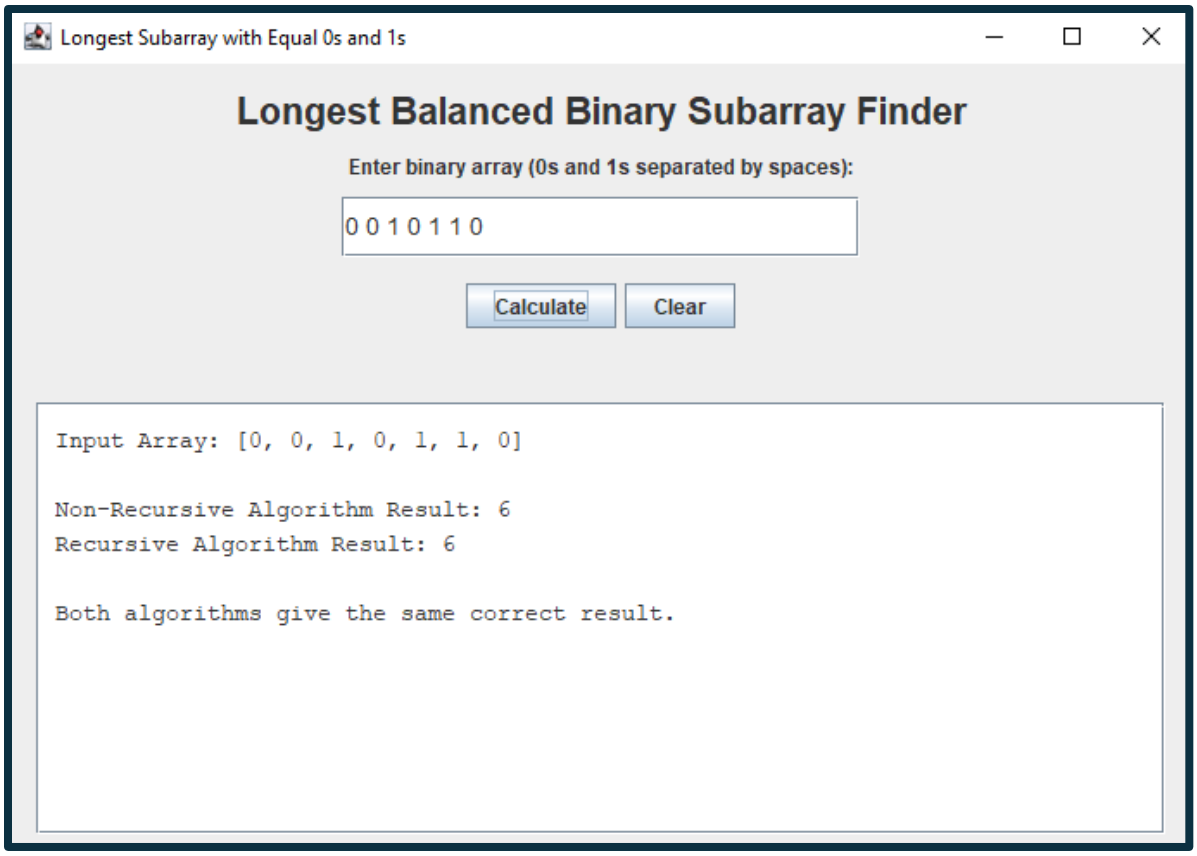
On the other hand, the recursive approach achieves the same logic but introduces additional overhead due to recursive function calls, which increases memory usage because of the call stack, especially for large input sizes.

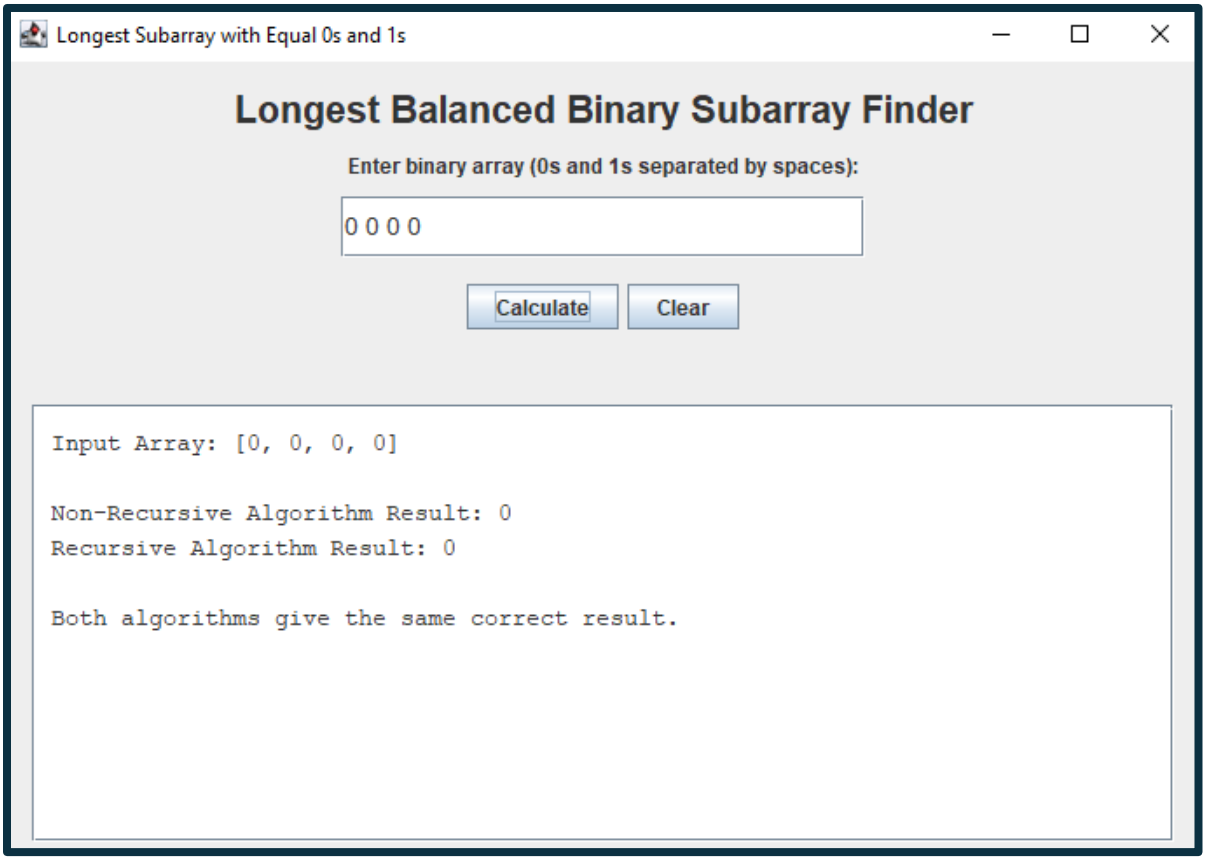
In conclusion, while both solutions are asymptotically equivalent, the non-recursive version is generally more efficient in practice due to lower runtime overhead and better memory utilization, making it more suitable for large datasets.

6- Examples:

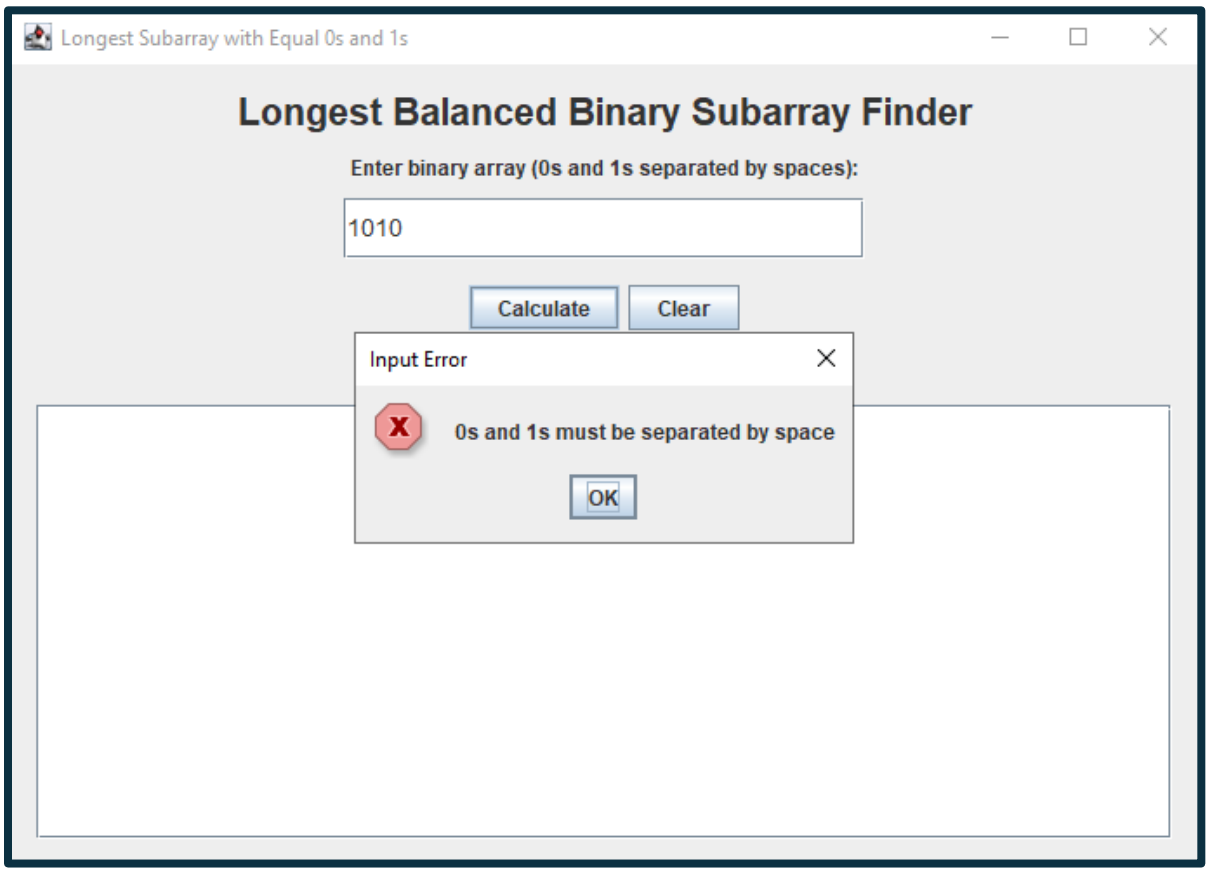

Longest Subarray with Equal 0s and 1s
Longest Balanced Binary Subarray Finder
Enter binary array (0s and 1s separated by spaces):
0 1
Calculate
Clear
Input Array: [0, 1]
Non-Recursive Algorithm Result: 2
Recursive Algorithm Result: 2
Both algorithms give the same correct result.

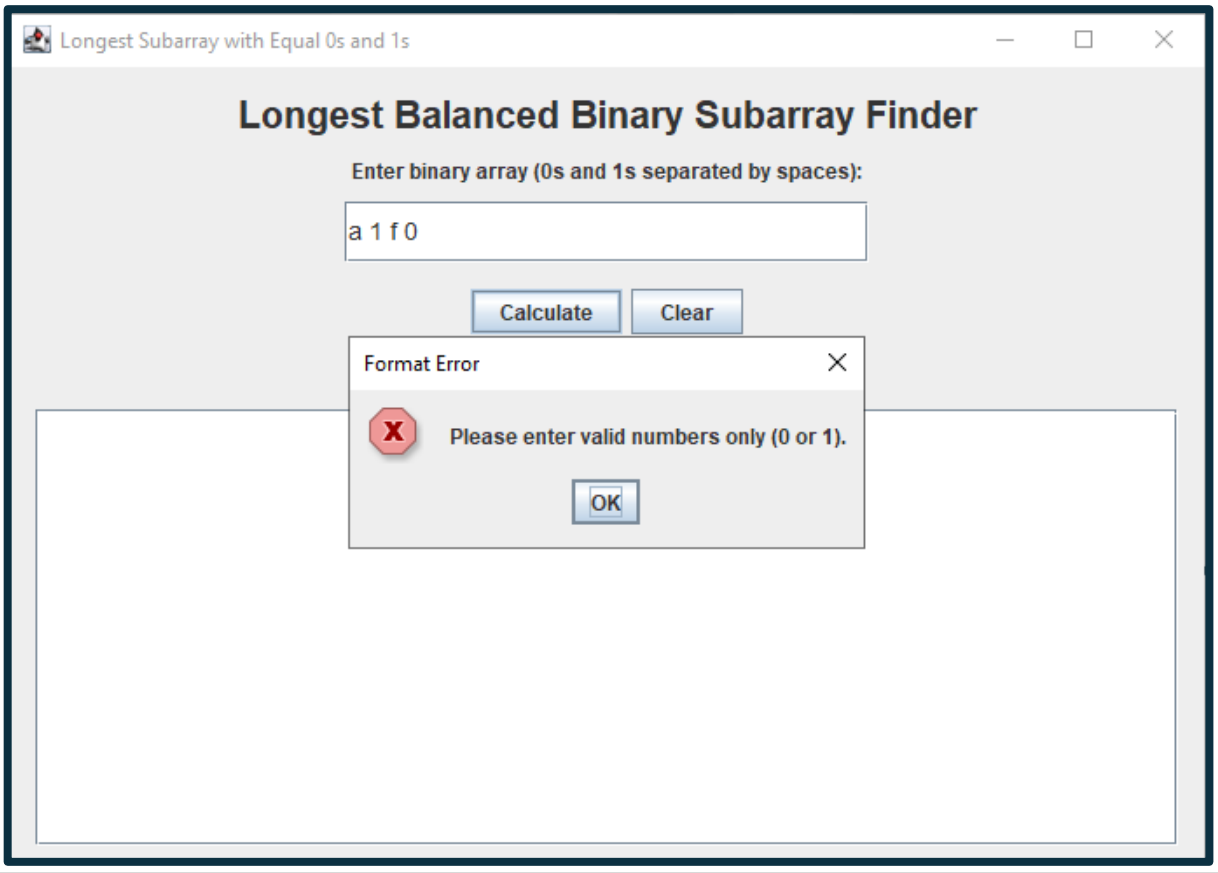

Longest Subarray with Equal 0s and 1s
Longest Balanced Binary Subarray Finder
Enter binary array (0s and 1s separated by spaces):
0 1 0
Calculate
Clear
Input Array: [0, 1, 0]
Non-Recursive Algorithm Result: 2
Recursive Algorithm Result: 2
Both algorithms give the same correct result.

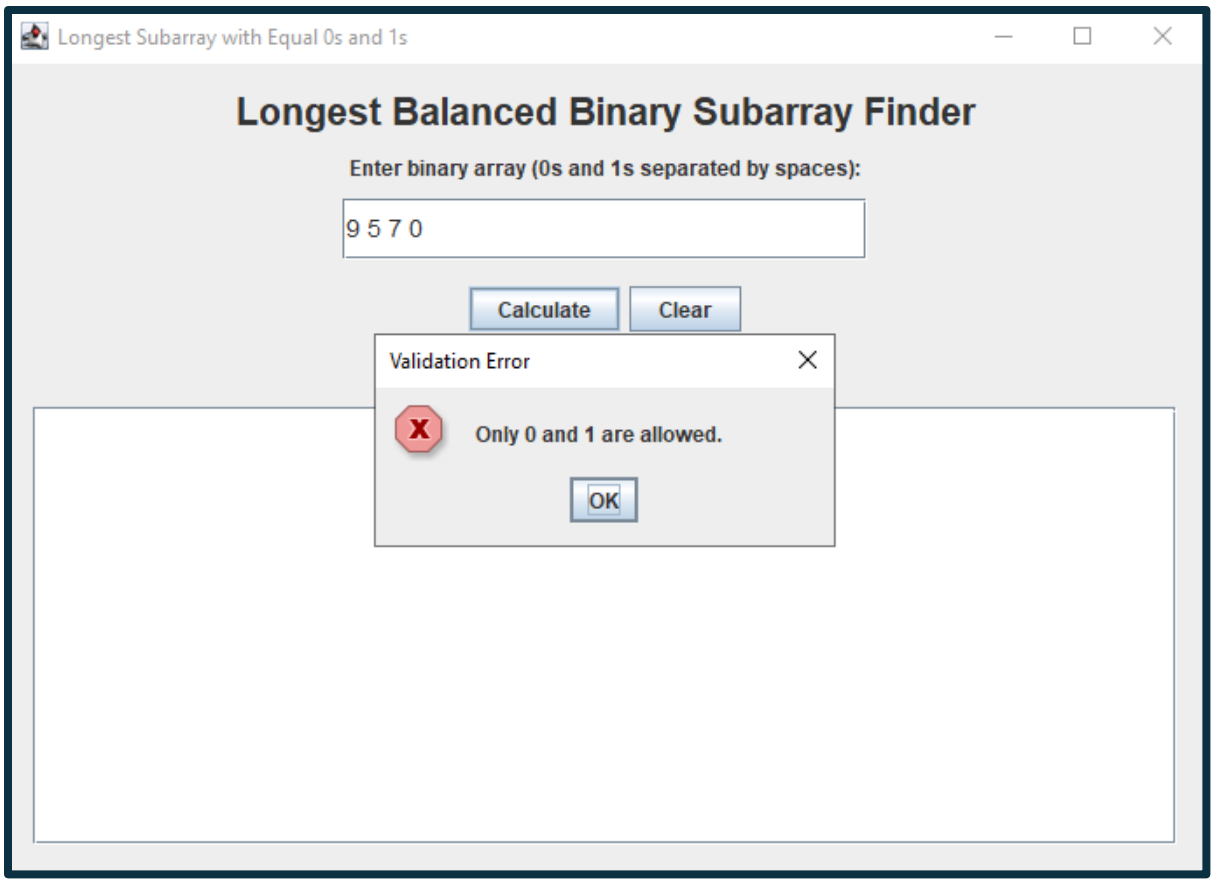

Longest Subarray with Equal 0s and 1s
Longest Balanced Binary Subarray Finder
Enter binary array (0s and 1s separated by spaces):
0 0 1 0 1 1 0
Calculate
Clear
Input Array: [0, 0, 1, 0, 1, 1, 0]
Non-Recursive Algorithm Result: 6
Recursive Algorithm Result: 6
Both algorithms give the same correct result.

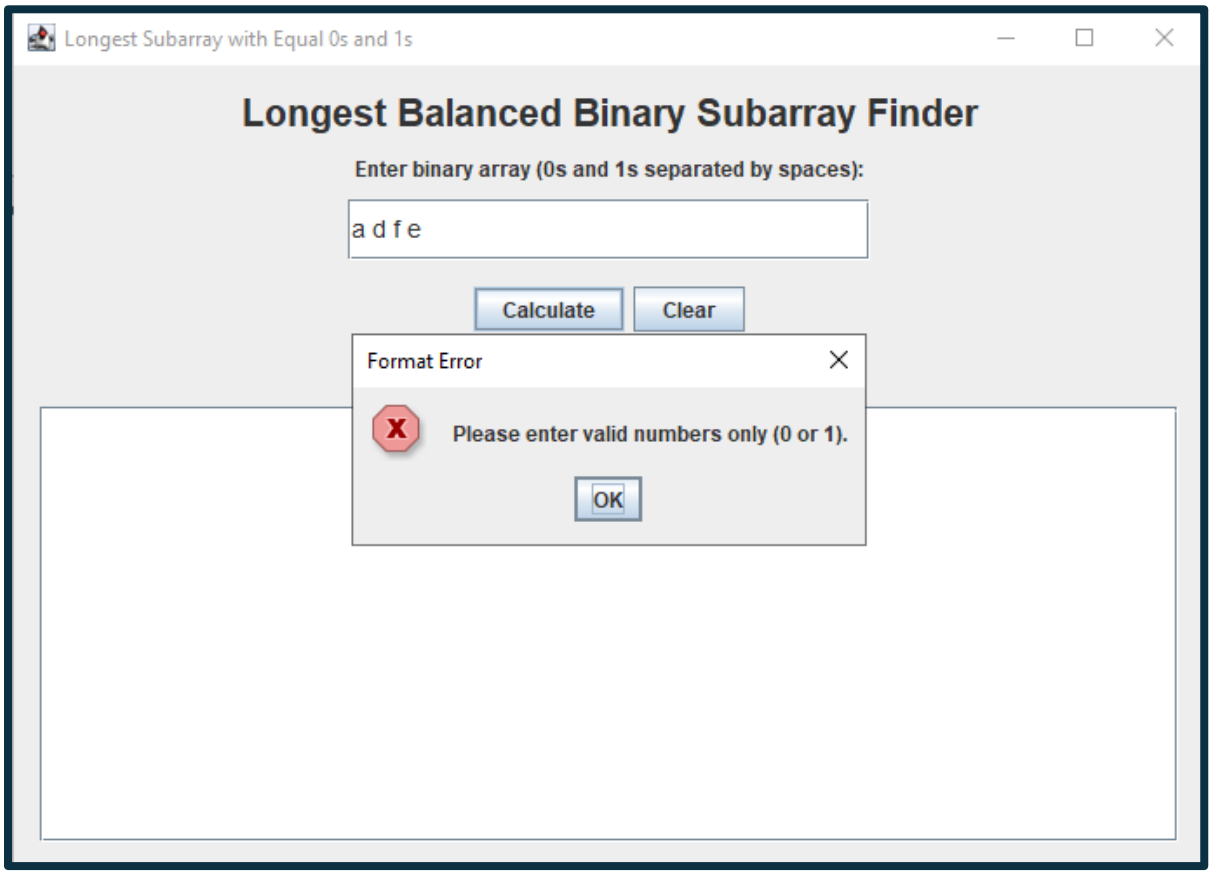

Longest Subarray with Equal 0s and 1s
Longest Balanced Binary Subarray Finder
Enter binary array (0s and 1s separated by spaces):
0 0 0 0
Calculate
Clear
Input Array: [0, 0, 0, 0]
Non-Recursive Algorithm Result: 0
Recursive Algorithm Result: 0
Both algorithms give the same correct result.

7- Validation:






Longest Subarray with Equal 0s and 1s
Longest Balanced Binary Subarray Finder
Enter binary array (0s and 1s separated by spaces):
9 5 7 0
Calculate
Clear
Validation Error
Only 0 and 1 are allowed.
OK


Longest Subarray with Equal 0s and 1s
Longest Balanced Binary Subarray Finder
Enter binary array (0s and 1s separated by spaces):
a d f e
Calculate
Clear
Format Error
Please enter valid numbers only (0 or 1).
OK